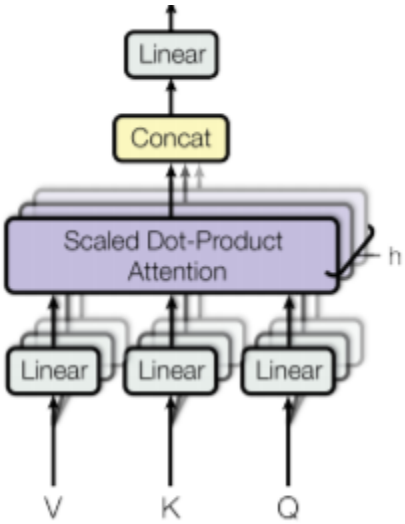
Understanding Transformer Architecture: Multi-Head Attention and Masked Attention Mechanisms

1. Introduction to Multi-Head Attention

Multi-head attention is a fundamental component introduced in the seminal paper “Attention Is All You Need.” This mechanism addresses a critical challenge in representing diverse objects and concepts within the same embedding space. The core insight behind multi-head attention lies in understanding that different contexts require different scales of representation.



1.1 The Scale Problem in Embeddings

When converting words into embeddings, each dimension of the vector represents a specific attribute. Consider the dimension representing size: an apple might have a size value of

approximately ten centimeters in this dimension. However, the same dimension must also accommodate vastly different scales—a hydrogen atom with an infinitesimally small measurement, or astronomical objects like black holes and galaxies with measurements in the trillions.

This creates a fundamental problem: when discussing space and celestial bodies like Mars, Earth, and Jupiter, we need to think in terms of planetary scales. Conversely, when discussing subatomic particles like hydrogen and helium atoms, we need to operate at an entirely different scale. The quality of the model's outputs depends critically on its ability to separate and distinguish objects that operate at similar scales.

1.2 Linear Transformations as Solution

The concept of linear transformation provides the solution to this scale problem. Drawing from computer graphics fundamentals, linear transformations allow us to manipulate vector representations in multiple ways. For instance, with a two-dimensional image, we can scale one dimension (the x-axis) independently—scaling it up makes the image wider, while scaling it down makes it thinner and taller.

This principle extends to n-dimensional vectors. Additionally, linear transformations include translation operations, where the origin of vectors can shift. By applying different linear transformations to the same input embeddings, we can view the same set of vectors from multiple perspectives. One transformation might focus on planetary-scale objects, while another focuses on atomic-scale objects.

2. Mathematical Implementation of Multi-Head Attention

2.1 Weight Matrices and Transformations

Mathematically, the fundamental attention formula remains unchanged—queries are still multiplied by keys. The n×n dimension matrix structure persists, but the process incorporates an

elegant modification. Between the standard attention operations, two weight matrices are introduced: the weight queries matrix and the weight keys matrix.

The first matrix has dimensions n×d (where n is the number of vectors and d is the embedding dimension). When multiplied by the query weights, this matrix must have d rows and k columns. The result is then multiplied by another weight matrix—the weight of keys. This keys matrix requires k rows (matching the number of columns in the first result) and d columns (matching the number of rows in the transpose of the keys).

Similarly, these transformed matrices are multiplied with a values matrix, which performs a linear transformation on the values embeddings. The complete result of multiplying all these matrices together produces the output for this particular linear transformation, known as an attention head.

2.2 Combining Multiple Heads

Since multiple linear transformations are applied (hence “multi-head” attention), each attention head processes the input through its own set of weight matrices for queries, keys, and values. Eventually, all outputs from the different attention heads are combined to produce a single unified output from the transformer attention block.

The expectation is that this combined output captures a comprehensive understanding of the context. When discussing planets and large-scale objects, the attention output should reflect this through larger values in the size dimension. Conversely, when discussing microscopic objects, the majority of attention heads will orient toward small-scale representations, and the output will reflect this focus.

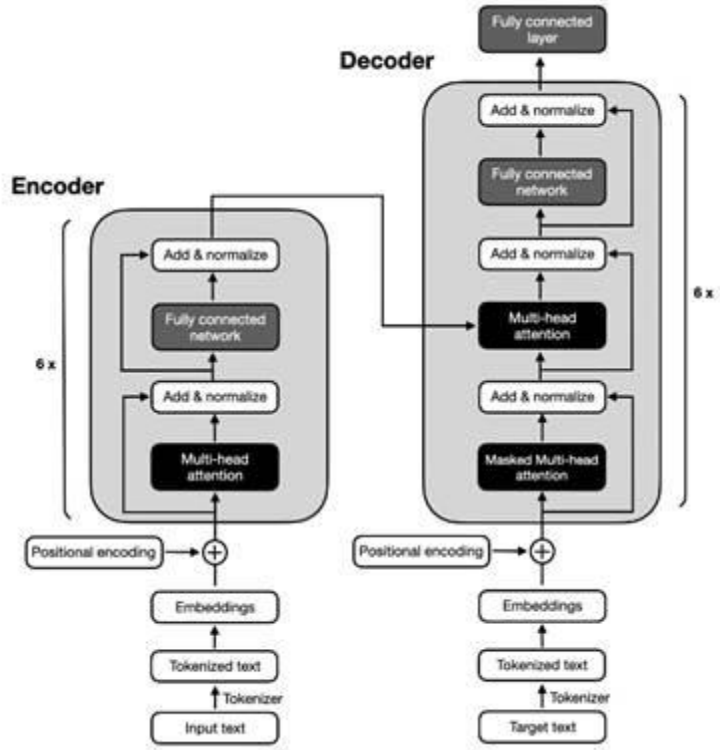
2.3 Trainable Parameters and Learning

A crucial advantage of this architecture is that the weight matrices themselves are trainable parameters. When the model makes mistakes during training, it’s not only the feed-forward layer weights that are adjusted through backpropagation—the weight matrices in the attention

mechanism are also updated. If the model produces errors, the values in these weight matrices change accordingly. Conversely, when predictions are accurate, the weights remain stable.

As the model trains on increasingly large datasets, these weight matrices continuously improve at identifying the appropriate linear transformations for different contexts. This adaptive learning enables the model to develop sophisticated strategies for representing objects at various scales and in various contexts.

3. Transformer Architecture Overview



3.1 Encoder-Decoder Structure

The complete transformer architecture, as presented in “Attention Is All You Need,” consists of an encoder on the left side and a decoder on the right side. However, the most popular implementation of transformers—GPT (Generative Pre-trained Transformer)—uses a purely

decoder architecture. Understanding GPT provides insight into how transformers work in their most common application.

In adapting the full transformer diagram to understand GPT, we remove the entire encoder component (the left-hand side). Additionally, GPT's architecture differs from the complete transformer in its attention blocks. While the full transformer shows masked multi-head attention, followed by multi-head attention, and then a feed-forward neural network, GPT eliminates the middle multi-head attention layer. This simplified structure makes GPT an ideal starting point for understanding transformer internals.

3.2 Input Processing and Position Encoding

At the bottom of the architecture, the process begins with output embeddings—the input sentence is first tokenized and then converted into vectors. These embeddings are enhanced with positional encoding, which incorporates information about each token's position within the sequence. This positional information is added directly to the embedding itself, allowing the model to understand not just what each word is, but where it appears in the sequence.

These position-encoded embeddings then flow into the masked multi-head attention block, where the most distinctive feature of the training process becomes apparent.

4. Masked Multi-Head Attention

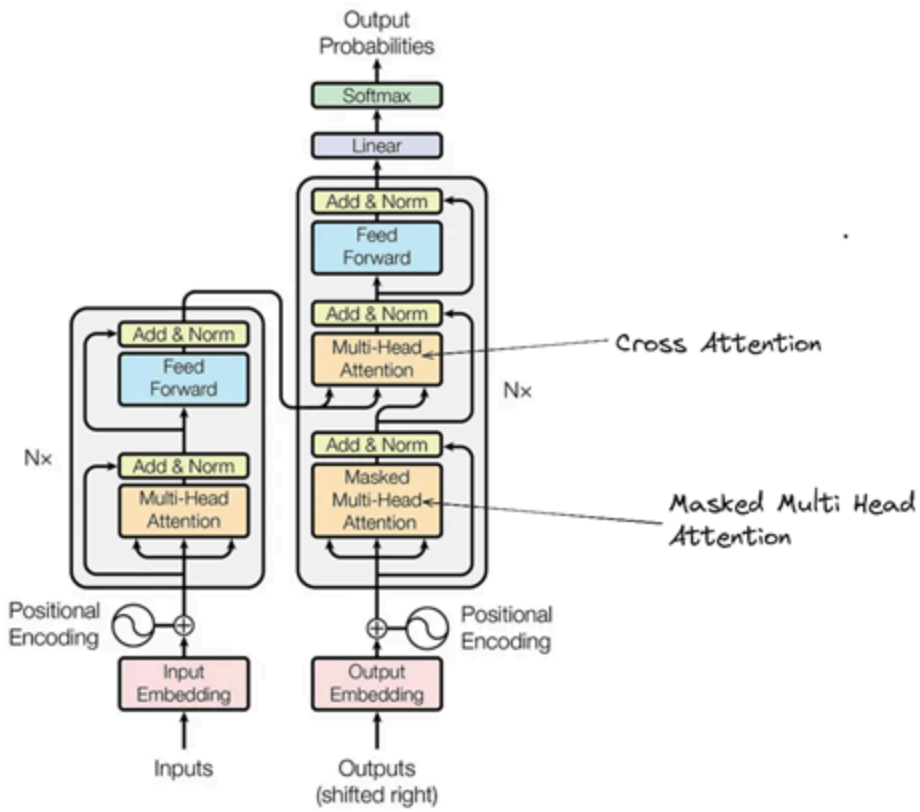


Figure 1: The Transformer - model architecture.

4.1 Training with Prefix Sequences

The masked attention mechanism implements a clever training strategy. Consider an original sentence: “my family member passed away.” The training process generates all possible prefix strings from this sentence. The prefixes are: “my,” “my family,” “my family member,” “my family member passed,” and “my family member passed away.”

The complete sentence (fifth prefix) is discarded as it provides nothing to predict. The remaining four prefixes become training examples. For the first sentence “my,” the model attempts to predict the next word. While “my cat,” “my dog,” or “my pen” are all possible, “my family” should also receive reasonable probability. If this probability is too low, the model is penalized through backpropagation. If the probability is appropriately high, no penalty is applied.

This process repeats for each prefix. For “my family,” the next word should show a reasonable probability for “member.” For “my family member,” the word “passed” should receive decent probability. Effectively, a single sentence is converted into multiple training inputs, enabling the model to learn the underlying structure of sentence construction.

4.2 Parallelization Through Masking

Processing each prefix individually would be extremely time-consuming. The innovation of masked attention allows for parallelization of this training process. Instead of processing prefixes sequentially, the original complete sentence is sent to the attention block, and values in the attention matrix that correspond to future positions are masked out.

In the attention matrix, for the first row (query “my”), only the key for “my” is visible. For the second row (query “family”), both “my” and “family” are visible as keys. For the third row (query “member”), “my,” “family,” and “member” are all visible. This creates a triangular or half-matrix pattern.

When the first row of the attention matrix multiplies with all columns of the value matrix, the resulting vector is influenced only by the first word. The second row, when multiplied with the value matrix, produces a vector influenced only by the first two words. Consequently, this second vector helps predict the word following “my family.”

This masking strategy allows the model to make multiple predictions simultaneously. From a sentence of five words, four predictions are generated in parallel by converting the full n^2 attention matrix into an effective $n^2/2$ matrix. This dramatically accelerates training by parallelizing what would otherwise be sequential operations.

4.3 Masking During Inference

An important distinction exists between training and inference. During training, masking is essential for the parallelization strategy described above. However, during inference—when a

user requests something like “write me an essay on artificial intelligence”—the masking step becomes unnecessary.

During inference, the model processes the input sequentially, generating one token at a time. There’s no need to create multiple prefix sequences or mask future positions because the model isn’t simultaneously predicting multiple next words. Each generation step only needs to attend to the tokens that have already been generated, making the masking mechanism redundant for inference, though the same attention structure is maintained.

5. Complete Information Flow

5.1 From Input to Output

The complete flow through a GPT-style transformer begins with tokenization of the input text. These tokens are converted to embeddings and enhanced with positional encoding. The position-encoded embeddings pass through the masked multi-head attention layer, where they are transformed through multiple attention heads, each applying different linear transformations via their respective weight matrices.

The outputs from all attention heads are concatenated and combined, producing a rich representation that captures different perspectives and scales of the input. This combined output then passes through a feed-forward neural network, which applies additional transformations. The final output represents the model’s prediction for the next token in the sequence.

5.2 Learning and Adaptation

Throughout training, when the model makes incorrect predictions, gradients flow backward through the entire network. These gradients update not only the feed-forward network weights but also all the weight matrices in the attention mechanism—the query weights, key weights, and value weights. Over time, with exposure to vast amounts of training data, these matrices

learn to perform increasingly sophisticated linear transformations that capture the nuances of language across different contexts and scales.

6. Key Concepts Summary

Multi-Head Attention Benefits

- Enables viewing the same input from multiple perspectives through different linear transformations
- Solves the scale problem by allowing different attention heads to focus on different scales (atomic vs planetary)
- Uses trainable weight matrices (W_Q, W_K, W_V) that improve during training
- Combines outputs from all heads to create a comprehensive representation

Masked Attention Benefits

- Converts single sentences into multiple training examples using prefix sequences
- Enables parallel training by masking future tokens in the attention matrix
- Creates a triangular attention pattern where each position only attends to previous positions
- Dramatically speeds up training while maintaining prediction quality

Training vs Inference

- **Training**: Masking is essential for parallelization and efficiency
- **Inference**: Masking is less critical as generation happens sequentially

7. Conclusion

The transformer architecture's success stems from its elegant combination of attention mechanisms with learnable linear transformations. Multi-head attention solves the scale problem inherent in representing diverse concepts within the same embedding space by viewing inputs through multiple learned perspectives. Masked attention enables efficient parallel training by converting single sentences into multiple training examples without sacrificing computational efficiency.

Together, these mechanisms create a powerful and flexible architecture capable of understanding and generating human language with remarkable sophistication. The trainable nature of all components ensures that the model continuously improves its representations as it processes more data, making transformers the foundation of modern large language models.